

Achieving Efficient and Privacy-Preserving Neural Network Training and Prediction in Cloud Environments

Chuan Zhang¹, Member, IEEE, Chenfei Hu¹, Tong Wu¹, Member, IEEE, Liehuang Zhu¹, Senior Member, IEEE, and Ximeng Liu², Senior Member, IEEE

Abstract—The neural network has been widely used to train predictive models for applications such as image processing, disease prediction, and face recognition. To produce more accurate models, powerful third parties (e.g., clouds) are usually employed to collect data from a large number of users, which however may raise concerns about user privacy. In this paper, we propose an Efficient and Privacy-preserving Neural Network scheme, named EPNN, to deal with the privacy issues in cloud-based neural networks. EPNN is designed based on a two-cloud model and techniques of data perturbation and additively homomorphic cryptosystem. This scheme enables two clouds to cooperatively perform neural network training and prediction in a privacy-preserving manner and significantly reduces the computation and communication overhead among participating entities. Through a detailed analysis, we demonstrate the security of EPNN. Extensive experiments based on real-world datasets show EPNN is more efficient than existing schemes in terms of computational costs and communication overhead.

Index Terms—Privacy-preserving, neural network, data perturbation, additively homomorphic cryptosystem, cloud environments

1 INTRODUCTION

RECENTLY, the neural network has received considerable attention as it can effectively extract features from high-dimensional data and build predictive models to connect the input data and output results. Due to its high accuracy, neural network has extensive applications in medical prediction[1], image recognition[2], product recommendation [3], etc. The amount of data plays a vital role in successful model training in neural networks, and thus many model requesters (e.g., individuals, companies) seek help from powerful third parties (e.g., clouds) to collect massive data from a large number of users and carry out neural network

training. With the trained models, clouds can also provide prediction services, helping users to make better decisions or see their full range of potential options. For example, a healthcare center can collect people's health data and use MLP to train prediction models for different diseases; a company can collect customers' online activities and use MLP to train models to provide more accurate recommendation services. This "training/prediction as a service" has greatly enhanced data acquiring and processing abilities, thereby opening a new door for neural networks.

In spite of the appealing benefits, data privacy concerns also arise for participating users [4], [5], [6]. In many neural network applications, users' data usually correlate with their private personal information, such as hobbies, genders, and health conditions. Without suitable protection, users may fear that their personal information will be leaked or abused. As a result, they may refuse to provide their data to the clouds for neural network training and prediction. In addition, the trained models are considered as model requesters' assets, and thus they do not want the models to be exposed to the clouds. In fact, it is dangerous for the clouds to know the models in plaintexts, as the clouds may recover sensitive information about the training data through a model inversion attack, as demonstrated in [7]. Therefore, to push forward cloud-based neural networks to flourish, we should enable the clouds to know nothing about the sensitive data and neural network models during the training and prediction processes.

Currently, increasing attention has been paid to the privacy issues in neural networks. Several privacy-preserving mechanisms have been proposed by [8], [9] to train neural network models over encrypted data in a third party. However, due to the utilization of high-cost encryption primitives

- Chuan Zhang, Tong Wu, and Liehuang Zhu are with the School of Cyber-space Science and Technology, Beijing Institute of Technology, Beijing 100811, China. E-mail: {chuanz, tongw, liehuangz}@bit.edu.cn.
- Chenfei Hu is with the School of Computer Science and Technology, Beijing Institute of Technology, Beijing 100811, China. E-mail: chenfeih@bit.edu.cn.
- Ximeng Liu is with the School of Information Systems, Singapore Management University, College of Mathematics and Computer Science, Fujian Provincial Key Laboratory of Information Security of Network Systems, Fuzhou University, Fuzhou, Fujian 350025, China. E-mail: snbnix@gmail.com.

Manuscript received 28 January 2022; revised 13 July 2022; accepted 14 September 2022. Date of publication 22 September 2022; date of current version 1 September 2023.

This work was supported in part by the National Natural Science Foundation of China under Grants 62202051 and 62102027, in part by the National Key Research and Development Program of China under Grant 2021YFB2700503, in part by China Postdoctoral Science Foundation under Grants 2021M700435, 2021TQ0042, and 2021TQ0041, in part by Shandong Provincial Key Research and Development Program under Grant 2021CXGC010106, in part by National Cryptography Development Fund under Grant MMJJ20180412, and in part by BIT Research and Innovation Promoting Project under Grant 2022YCXZ031.

(Corresponding authors: Tong Wu; Liehuang Zhu.)

Digital Object Identifier no. 10.1109/TDSC.2022.3208706

(e.g., fully homomorphic encryption (FHE) or garbled circuit), these schemes are computationally expensive to be deployed in practical applications. Yuan et al. [10] presented a server-aid machine learning framework, where trainers (i.e., data owners) that hold an arbitrary part of datasets perform neural network model training with the help of a third server. Thanks to an efficient cryptographic tool [11], their scheme achieves high computational efficiency. However, the communication efficiency issue remains since the scheme [10] still requires frequent communication between the server and data owners. The above schemes are all interactive approaches, in which data owners have to interact with the server to update the neural network models. These interactive approaches may not be practical in real-world applications since (1) the data owners may not always be online; (2) frequent interactions will introduce a significant computation and communication overhead to the data owners.

Therefore, some recent works have been devoted to designing non-interactive privacy-preserving neural network training and prediction schemes. Mohassel et al. [12] presented a scalable machine learning scheme by employing two non-colluding cloud servers. Using garbled circuit, oblivious transfer, and Beaver's multiplication triplet, their method can support neural network models to be privately trained in the clouds without data owners' involvement. Some enhanced schemes, such as [13], [14], are then presented to improve the efficiency of [12]. Although many efforts have been devoted, the above schemes are designed based on the garbled circuit and oblivious transfer, leading to huge computation and communication overhead between the two clouds with the increasing iterations of multiplications. Recently, Li et al. [15] proposed a Non-Interactive Privacy-Preserving Multi-Party Machine Learning (NPMML) scheme with satisfactory efficiency on data owners' side. In their scheme, data owners only act as data providers and do not participate in the subsequent model training process. However, to protect the model privacy, model requesters still need to cooperate with clouds to train the models. Furthermore, some other schemes are proposed for privacy-preserving neural network prediction [16], [17], [18]. These schemes, however, either cannot support non-interactive neural network training or fall short of achieving non-interactive neural network prediction.

In this paper, to deal with the aforementioned security and privacy challenges, we design a novel non-interactive neural network scheme to support efficient and privacy-preserving neural network training and prediction. Specifically, we employ two non-colluding clouds to cooperatively train the neural network model and provide prediction services. During the training and prediction procedures, the users (i.e., data owners, model requesters, and model users) are not involved in the iteration process. Our contributions can be summarized as follows:

- We propose an efficient and privacy-preserving neural network training and prediction scheme (EPNN) in cloud environments. EPNN enables privacy-preserving neural networks to be performed on two non-colluding clouds efficiently, without any user involved in the training and prediction processes.

- We construct a data perturbation technique to protect the model privacy and employ an additively homomorphic cryptosystem to protect the data privacy. Then, we further design an efficient interaction process on two clouds to train neural network models and provide prediction services, where all calculations are performed in a privacy-preserving manner.
- We present a comprehensive security analysis on EPNN in terms of data privacy and model privacy. We implement EPNN in Python and evaluate the performance using real-world datasets. The experimental results demonstrate the EPNN is of great accuracy and high efficiency. Specifically, compared with the state-of-the-art non-interactive neural network training schemes, EPNN brings up at least $16.2 \times$ improvement in computational efficiency and 50% savings in communication overhead on the dataset of MNIST.

Organization. The remainder of this paper is organized as follows. We present the system/threat model and design goals of EPNN in Section 2. In Section 3, we review the technique of multi-layer perception and the cryptographic tool used in our scheme. Section 4 introduces the details of our scheme and Section 5 gives its security analysis. Performance evaluation is presented in Section 6. In Section 7, we review the related works and our work is concluded in Section 8.

2 MODELS AND DESIGN GOALS

In this section, we first describe the system and threat model of EPNN. Then, we introduce the design goals of EPNN to clarify the problems to be resolved.

2.1 System Model

The system model mainly consists of five entities: trusted third party (referred to as TTP for short), cloud service provider (referred to as CSP for short), the model requester, the data owner, and the model user. The system model is shown in Fig. 1, and each entity is described as follows.

- TTP is responsible for generating key pairs based on security parameters. It then distributes the secret key to two CSPs and publishes the public key to all participants. After that, TTP will be offline.
- CSP is a powerful third party with great computation and storage capabilities. To free users from the time-consuming neural network training process and to help users conveniently enjoy neural network prediction services, the CSP is usually introduced to undertake the model training and prediction task.
- The model requester is usually an individual or an organization who wants to obtain neural network models for various applications (e.g., disease prediction, face recognition, and risk assessment). Since the model requester usually pays for the model training services, the trained models can be considered as the private asset of the model requester. Thus, the trained models should not be obtained by other participants. Considering that the model requester is

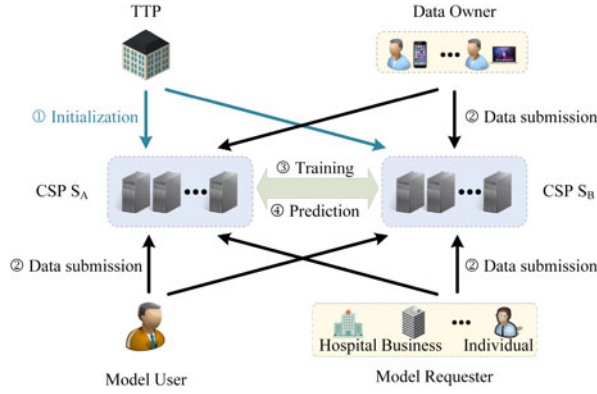


Fig. 1. System model.

usually resource-constrained, he/she is not supposed to participate in the training phase.

- The data owner is an entity that possesses a set of data for neural network training. To protect data privacy, the data owner should encrypt his/her training data and upload the ciphertexts to the CSPs. Similar to the model requester, the data owner is also expected to stay offline after data submission.
- The model user is an entity that requests neural network prediction services. To protect data privacy, the model user encrypts his/her prediction data (i.e., data to be predicted) and uses the trained models to obtain prediction results.

2.2 Threat Model

In this paper, we mainly consider privacy threats from the internal malicious parties, since they are more destructive than external adversaries.

All entities follow the *honest-but-curious* model, in which the entities are honest in performing the designed operations but attempt to obtain other parties' private information. In addition, we assume that the participants will not collude with each other, which is a common assumption in most two-cloud-based applications [12], [17], [19], [20]. Here, we describe the four types of adversaries in our proposed EPNN as follows:

- *Type-I: CSP adversary.* The CSP is responsible for training neural network models based on encrypted training data and perturbed initial models received from the data owner and the model requester, respectively, providing prediction servers to the model user. By compromising the semi-honest CSP, the adversary can obtain the encrypted training/prediction data, perturbed models and intermediate calculation results. The adversary expects to infer sensitive information about training/prediction data and models.
- *Type-II: Model requester adversary.* The model requester is offline after submitting the perturbed initial models. After model training, he/she can obtain the trained models from the CSPs. By compromising the semi-honest model requester, the adversary can obtain the initial model, the trained model and expects to infer sensitive information about the training data.

- *Type-III: Data owner adversary.* The data owner is offline after submitting the encrypted training data. By compromising the semi-honest data owner, the adversary can obtain the training data and expects to infer sensitive information about the models and others' training data.
- *Type-IV: Model user adversary.* The model user is offline after submitting the encrypted prediction data. After model prediction, he/she can obtain prediction results from the CSPs. By compromising the semi-honest model user, the adversary can obtain the prediction data, the prediction results and expects to infer sensitive information about the models.

2.3 Design Goals

Under the above system and threat models, the following design goals should be achieved:

- *Data/Model Privacy during Neural Network Training.* Suppose there is a collection of data records $\{\mathbf{x}_m, \mathbf{y}_m\}_{m=1}^M$ that is sent to CSPs for model training, where $\mathbf{x}_m$ is the training data, $\mathbf{y}_m$ is its corresponding labels. Here, we use $\mathbf{W}_h, \mathbf{W}_o$ to denote the model weights of hidden layer and output layer, respectively. Our goal is to estimate $\mathbf{W}_h, \mathbf{W}_o$ based on the training data while protecting the training data and models from being disclosed to other entities.
- *Data/Model Privacy during Neural Network Prediction.* Suppose there is a piece of data $\mathbf{x}$ and this data is sent to CSPs for prediction. Our goal is to estimate the prediction results using the trained models while not disclosing the privacy of trained models, prediction data and prediction results.
- *Efficiency.* Considering that most users are usually resource-restrained and may not always be online, we require that there should be no interactions between the users and CSPs in the above two phases. In other words, users only need to submit their data once during the whole system procedure. In addition, the computational costs incurred on the clouds should be as minimal as possible.

3 PRELIMINARIES

This section reviews the concepts of multi-layer perception (MLP) and additively homomorphic cryptosystem, which serve as the basis of our proposed scheme.

3.1 Multi-Layer Perception

Multi-layer perception (MLP) is one of the most popular architectures in neural networks. It consists of three layers (i.e., input layer, hidden layer, and output layer), and it aims to build predictive models to connect the inputs and outputs. If we use a to denote the dimension of the training data, b to denote the number of nodes in the hidden layer, and c to denote the dimension of the output layer, the training process of a typical MLP can be presented as follows.

- *Step ①:* Given a set of inputs $\{\mathbf{x}_i \in \mathbb{R}^a, \mathbf{y}_i \in \mathbb{R}^c\}_{i=1}^M$, randomly generating the model weights of hidden layer $\mathbf{W}_h^{(0)} \in \mathbb{R}^{b \times a}$ and output layer $\mathbf{W}_o^{(0)} \in \mathbb{R}^{c \times b}$.

- *Step ②*: In the t th iteration, the hidden layer receives the data from the input layer and calculates the weighted data $\mathbf{W}_h^{(t)} \mathbf{x}$ with current model weights of hidden layer $\mathbf{W}_h^{(t)}$. Then, the hidden layer outputs $\mathbf{h} = f(\mathbf{W}_h^{(t)} \mathbf{x})$, where $f(\cdot)$ is an activation function.
- *Step ③*: With the value $\mathbf{h}$ calculated by the hidden layer, the output layer computes the output value $\mathbf{o} = \mathbf{W}_o^{(t)} \mathbf{h}$ with current model weights of output layer $\mathbf{W}_o^{(t)}$. Then, it calculates the difference between the label $\mathbf{y}$ and $\mathbf{o}$, and based on this updates the models weights of hidden layer and output layer as $\mathbf{W}_h^{(t+1)}$ and $\mathbf{W}_o^{(t+1)}$.
- *Step ④*: Repeating *Step 2* and *Step 3* until the convergence criterion (e.g., the maximum iteration number or the difference between two consecutive models is less than a threshold value) is achieved.

Learning the models of MLP is a non-linear optimization problem and it is usually solved by gradient descent, an optimization algorithm to find the minimum of a function. Among the various gradient descent algorithms, stochastic gradient descent (SGD) is a simple but effective approach to calculate the gradient over a small-batch of the whole training data. Let J be the loss function, $h_{j,i}$, $\mathbf{wh}_{j,i}^{(t)}$, $\mathbf{wo}_{k,j}^{(t)}$ be the elements in $\mathbf{h}$, $\mathbf{W}_h^{(t)}$, $\mathbf{W}_o^{(t)}$, respectively, SGD calculates the partial derivatives of J with respect to the model weights of hidden layer and output layer in the t th iteration as $\frac{\partial J}{\partial \mathbf{wh}_{j,i}^{(t)}}$, $\frac{\partial J}{\partial \mathbf{wo}_{k,j}^{(t)}}$, respectively. Accordingly, the models weights of the hidden layer and output layer in the $(t+1)$ th iteration are updated as

$$\begin{aligned} \mathbf{wh}_{j,i}^{(t+1)} &= \mathbf{wh}_{j,i}^{(t)} - \eta \frac{\sum_S \frac{\partial J}{\partial \mathbf{wh}_{j,i}^{(t)}}}{S}, \\ \mathbf{wo}_{k,j}^{(t+1)} &= \mathbf{wo}_{k,j}^{(t)} - \eta \frac{\sum_S \frac{\partial J}{\partial \mathbf{wo}_{k,j}^{(t)}}}{S}, \end{aligned} \quad (1)$$

where S is the batch size and η is the learning rate.

3.2 Additively Homomorphic Cryptosystem

The additively homomorphic cryptosystem used in this paper is Paillier cryptosystem with threshold decryption (PCTD)[21]. PCTD divides the secret key into several parts, and the ciphertexts can be decrypted only if all key holders cooperate with each other. Due to its high efficiency and scalability, PCTD is widely adopted in secure multi-party computation applications. In the following, we list PCTD's several key algorithms that are used in this work. Interested readers can refer to [21] for its full version.

- $\text{Gen}(\kappa)$: On input a security parameter κ , this algorithm first selects two large prime numbers p, q with the same κ bits, and computes $N = pq$, $\lambda = \text{lcm}(p-1, q-1)$, where lcm denotes lowest common multiple. Then, this algorithm selects a generator g with order $(p-1)(q-1)/2$, a random value $sk \in \mathbb{Z}_N$, and outputs the public key and secret key as

Public Key: $(pk = g^{sk} \bmod N^2, N)$,

Secret Key: $sk = \lambda$.

- $\text{Enc}(pk, m)$: This algorithm selects a random value $r \in [1, N/4]$ to encrypt a message $m \in \mathbb{Z}_N$ as $[m] = pk^r(1 + mN) \bmod N^2$.
- $\text{KeyS}(\lambda, k)$: This algorithm randomly splits the secret key λ into k parts $(\lambda_1, \dots, \lambda_k)$ such that $\sum_{i=1}^k \lambda_i \equiv 0 \bmod \lambda$ and $\sum_{i=1}^k \lambda_i \equiv 1 \bmod N^2$.
- $\text{PD}(\lambda_i, [m])$: This algorithm uses a part of the secret key λ_i to partially decrypt the ciphertexts as $[m]^{\lambda_i}$.
- $\text{DD}(\{[m]^{\lambda_i}\}_{i=1}^k)$: On input of the partially decrypted ciphertexts, this algorithm outputs the plaintexts by computing $m = L(\prod_{i=1}^k [m]^{\lambda_i})$, where $L(x) = \frac{x-1}{N}$.

As an additively homomorphic cryptosystem, PCTD holds the homomorphic properties

$$\begin{aligned} [m_1] \cdot [m_2] &= [m_1 + m_2], \\ [m_1]^a &= [a \cdot m_1], \end{aligned}$$

where $\{m_1, m_2, a\} \in \mathbb{Z}_N$. Note that, in PCTD cryptosystem, $[-m]$ is obtained by computing $[m]^{N-1}$. For easy understanding, we use $[m]^{-1}$ to denote $[m]^{N-1}$ in the following sections.

4 PROPOSED SCHEME

The main process of EPNN includes *initialization*, *data submission*, *neural network training*, and *neural network prediction*, which is illustrated in Fig. 1 and briefly summarized below.

- *Initialization*. In the initialization, given security parameters, the TTP executes the PCTD cryptosystem to generate the key pairs. The secret key is sent to two CSPs, while the public key is published to all participants.
- *Data Submission*. In this phase, the model requester perturbs his/her models using random values, while the data owner and model user encrypt their data using the public key. Then, they submit the encrypted data or perturbed models to CSPs for neural network training and prediction. After that, they can keep offline until the results are returned.
- *Neural Network Training*. Based on the encrypted data and perturbed models, two CSPs securely and cooperatively train the hidden/output layer models of MLP. After training, CSPs return the perturbed final models to the model requester.
- *Neural Network Prediction*. With the trained models, two CSPs execute privacy-preserving neural network prediction to get the prediction results for the model user. Finally, the perturbed prediction results are sent to the model user.

Then, we give the detailed construction of EPNN. For ease of presentation, we denote each data owner as DO, the two CSPs as S_A, S_B , the model requester as MR, and the model user as MU.

4.1 Initialization

As mentioned above, an additively homomorphic cryptosystem PCTD = $\{\text{Gen}(\kappa), \text{Enc}(pk, m), \text{KeyS}(\lambda, k), \text{PD}(\lambda_i, [m]), \text{DD}(\{[m]^{\lambda_i}\}_{i=1}^k)\}$ is used in the scheme, where the κ is a security parameter. The initialization steps are shown as follows.

Step ①: TTP executes $\text{Gen}(\kappa)$ to generate public key (pk, N) and secret key λ and then executes $\text{KeyS}(\lambda, 2)$ to splits the secret key λ to two parts (λ_A, λ_B) .

Step ②: TTP distributes the secret keys λ_A, λ_B to $\mathcal{S}_A, \mathcal{S}_B$, respectively, and then publishes the public key (pk, N) to all participants.

4.2 Data Submission

This phase is conducted only once during the whole procedure of EPNN.

Step ①: A model requester MR initializes model weights of the hidden layer and output layer as $\mathbf{W}_h^{(0)} \in \mathbb{Z}_\lambda^{b \times a}, \mathbf{W}_o^{(0)} \in \mathbb{Z}_\lambda^{c \times b}$ and perturbs them by selecting random values $\{\alpha_{j,i}\}_{j,i=1}^{b,a}, \{\beta_{k,j}\}_{k,j=1}^{c,b}$ as

$$\begin{aligned}\widetilde{\mathbf{wh}}_{j,i}^{(0)} &= \mathbf{wh}_{j,i}^{(0)} + \alpha_{j,i}, \\ \widetilde{\mathbf{wo}}_{k,j}^{(0)} &= \mathbf{wo}_{k,j}^{(0)} + \beta_{k,j},\end{aligned}$$

where the random values are chosen from $\mathbb{Z}_\lambda$. After that, MR encrypts the random values as $[\alpha_{j,i}], [\beta_{k,j}]$, and sends $(\{\widetilde{\mathbf{wh}}_{j,i}^{(0)}\}_{j,i=1}^{b,a}, \{\beta_{k,j}\}_{k,j=1}^{c,b})$ to $\mathcal{S}_A$, $(\{\widetilde{\mathbf{wo}}_{k,j}^{(0)}\}_{k,j=1}^{c,b}, \{\alpha_{j,i}\}_{j,i=1}^{b,a})$ to $\mathcal{S}_B$ via secure communication channels.

Step ②: Each data owner DO uses the public key to encrypt his/her data as $\{[x_{m,i}]\}_{i=1}^a, \{[y_{m,k}]\}_{k=1}^c$, divides $x_{m,i}$ into two random parts $x_{m,(1,i)}, x_{m,(2,i)}$ such that $x_{m,i} = x_{m,(1,i)} \cdot x_{m,(2,i)}$, and further calculates $\{\prod_{i=1}^a [\alpha_{j,i}]^{-x_{m,i}}\}_{j=1}^b$. Then, the ciphertexts $\{[x_{m,i}]\}_{i=1}^a, \{\prod_{i=1}^a [\alpha_{j,i}]^{-x_{m,i}}\}_{j=1}^b$ and the plaintexts $\{x_{m,(1,i)}\}_{i=1}^a$ are sent to $\mathcal{S}_A$, while the data $\{[y_{m,k}]\}_{k=1}^c, \{x_{m,(2,i)}\}_{i=1}^a$ are sent to $\mathcal{S}_B$.

Step ③: For the data $\mathbf{x}_u = [x_{u,1}, \dots, x_{u,a}]$ needed to be predicted, MU selects random values $\mathbf{r}_u = [r_{u,1}, \dots, r_{u,c}]$, where $\{r_{u,i} \in \mathbb{Z}_\lambda\}_{i=1}^c$, and calculates $[\mathbf{x}_u], [\mathbf{r}_u], \{\prod_{i=1}^a [\alpha_{j,i}]^{-x_{u,i}}\}_{j=1}^b$. After that, MU sends the ciphertexts $([\mathbf{x}_u], \{\prod_{i=1}^a [\alpha_{j,i}]^{-x_{u,i}}\}_{j=1}^b, [\mathbf{r}_u])$ to $\mathcal{S}_A, \mathcal{S}_B$, respectively.

It is possible that the data needed to be encrypted may not be integers. In this case, a parameter L , which is a magnitude of 10, is used to multiply the original data and at last the accurate results can be recovered by dividing the parameter L . Additionally, if $x_{m,i} = 0$, DO can select two very small values $x_{m,(1,i)}, x_{m,(2,i)}$ such that $x_{m,(1,i)} \cdot x_{m,(2,i)} \approx 0$.

4.3 Neural Network Training

This phase is iteratively conducted between the two CSPs. Specifically, the following steps are taken at the t th iteration.

Step ①: With the encrypted data and the perturbed model weights of hidden layer, $\mathcal{S}_A$ selects a random S -size data S_t from the whole encrypted dataset. Then, for each data record in S_t , $\mathcal{S}_A$ first calculates the weighted data in ciphertexts as

$$\begin{aligned}C_{m,j} &= \prod_{i=1}^a [x_{m,i}]^{\widetilde{\mathbf{wh}}_{j,i}^{(t)}} \cdot \prod_{i=1}^a [\alpha_{j,i}]^{-x_{m,i}} \\ &= \left[\sum_{i=1}^a x_{m,i} \mathbf{wh}_{j,i}^{(t)} \right],\end{aligned}$$

and then partially decrypts $C_{m,j}$ with its secret key as

$$\text{PD}(\lambda_A, C_{m,j}) \rightarrow C_{m,j}^{\lambda_A}.$$

After that, $\mathcal{S}_A$ sends $\{C_{m,j}, C_{m,j}^{\lambda_A}\}_{m,j=1}^{S,b}$ to $\mathcal{S}_B$.

Step ②: Upon receiving the ciphertexts, $\mathcal{S}_B$ recovers $\sum_{i=1}^a x_{m,i} \mathbf{wh}_{j,i}^{(t)}$ by performing $\text{PD}(\lambda_B, C_{m,j})$ and $\text{DD}(C_{m,j}^{\lambda_A}, C_{m,j}^{\lambda_B})$. Then, $\mathcal{S}_B$ calculates the hidden layer $\mathbf{h}_m = (h_{m,1}, \dots, h_{m,b})$, where each element in $\mathbf{h}_m$ is computed as

$$h_{m,j} = f\left(\sum_{i=1}^a x_{m,i} \mathbf{wh}_{j,i}^{(t)}\right), j \in [1, b].$$

Here, we select sigmoid as the activation function and thus we have $h_{m,j} = \frac{1}{1+e^{-\sum_{i=1}^a x_{m,i} \mathbf{wh}_{j,i}^{(t)}}}$. After that, $\mathcal{S}_B$ calcu-

lates the output values (i.e., prediction results) in ciphertexts, i.e., $[o_{m,k}], m \in [1, S], k \in [1, c]$, by

$$\begin{aligned}[o_{m,k}] &= \prod_{j=1}^b [\widetilde{\mathbf{wo}}_{k,j}^{(t)}]^{h_{m,j}} \cdot [\beta_{k,j}]^{-h_{m,j}} \\ &= \left[\sum_{j=1}^b \mathbf{wo}_{k,j}^{(t)} h_{m,j} \right].\end{aligned}$$

With $[o_{m,k}]$, $\mathcal{S}_B$ computes the difference between the labels and the prediction results as

$$\begin{aligned}\zeta_{m,k} &= ([y_{m,k}] \cdot [o_{m,k}]^{-1}) \\ &= [y_{m,k} - o_{m,k}],\end{aligned}$$

and then calculates the partial derivative with respect to the model weights of output layer as

$$\begin{aligned}\delta_{m,(k,j)} &= \zeta_{m,k}^{-h_{m,j}} \\ &= [-(y_{m,k} - o_{m,k}) h_{m,j}].\end{aligned}$$

Accordingly, the model weights of output layer is updated as

$$\begin{aligned}[\widetilde{\mathbf{wo}}_{k,j}^{(t+1)}] &= [\widetilde{\mathbf{wo}}_{k,j}^{(t)}] \cdot \left(\prod_{m=1}^S \delta_{m,(k,j)} \right)^{-\eta/S} \\ &= \left[\mathbf{wo}_{k,j}^{(t)} + \beta_{k,j} - \eta \frac{\sum_{m=1}^S -(y_{m,k} - o_{m,k}) h_{m,j}}{S} \right] \\ &= [\mathbf{wo}_{k,j}^{(t+1)} + \beta_{k,j}].\end{aligned}$$

Then, the ciphertexts $\{[\widetilde{\mathbf{wo}}_{k,j}^{(t+1)}]\}_{k,j=1}^{c,b}, \{\zeta_{m,k}\}_{m,k=1}^{S,c}$ and $\{\prod_{k=1}^c \widetilde{\mathbf{wo}}_{k,j}^{(t+1)}\}_{m,j=1}^{S,b}$ are sent to $\mathcal{S}_A$.

Step ③: On receiving the ciphertexts from $\mathcal{S}_B$, $\mathcal{S}_A$ computes intermediate results of the partial derivative with respect to the model weights of hidden layer as

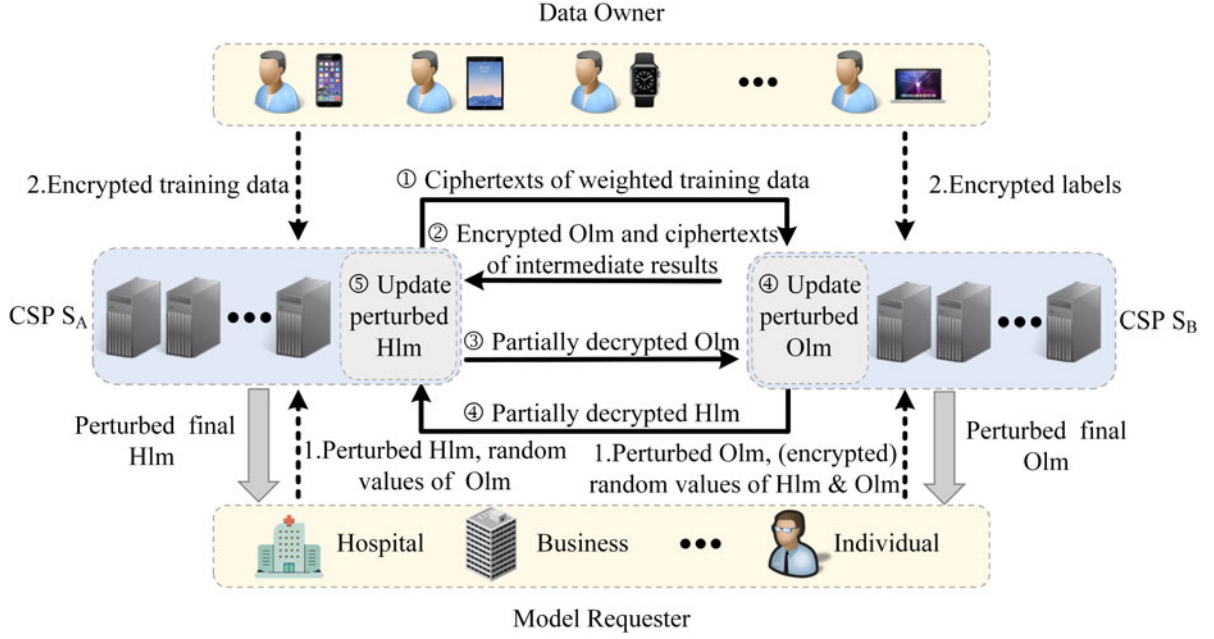


Fig. 2. Training process of EPNN, where steps 1-2 are in the data submission phase, steps ①-⑤ are in the neural network training phase, Hlm refers to the hidden layer model, and Olm refers to the output layer model.

$$\begin{aligned}\Delta_{m,(j,i)} &= \left(\prod_{k=1}^c \zeta_{m,k}^{\tilde{\mathbf{w}}_{k,j}^{(t)}} \prod_{k=1}^c \zeta_{m,k}^{-\beta_{k,j}} \right)^{x_{m,(1,i)}} \\ &= \left(\prod_{k=1}^c \zeta_{m,k}^{\tilde{\mathbf{w}}_{k,j}^{(t)} - \beta_{k,j}} \right)^{x_{m,(1,i)}} \\ &= \left[x_{m,(1,i)} \sum_{k=1}^c (y_{m,k} - o_{m,k}) \mathbf{w}_{k,j}^{(t)} \right].\end{aligned}$$

Then, it partially decrypts the ciphertexts of output layer via $\text{PD}(\lambda_A, [\tilde{\mathbf{w}}_{k,j}^{(t+1)}]) \rightarrow [\tilde{\mathbf{w}}_{k,j}^{(t+1)}]^{\lambda_A}$. At last, $\mathcal{S}_A$ sends

$$\{\Delta_{m,(j,i)}\}_{m,j,i=1}^{S,b,a}, \{[\tilde{\mathbf{w}}_{k,j}^{(t+1)}]^{\lambda_A}\}_{k,j=1}^{c,b} \text{ to } \mathcal{S}_B.$$

Step ④: In this step, $\mathcal{S}_B$ uses its secret key to recover the model weights of output layer by performing $\text{PD}(\lambda_B, [\tilde{\mathbf{w}}_{k,j}^{(t+1)}]) \rightarrow [\tilde{\mathbf{w}}_{k,j}^{(t+1)}]^{\lambda_B}$, $\text{DD}([\tilde{\mathbf{w}}_{k,j}^{(t+1)}]^{\lambda_A}, [\tilde{\mathbf{w}}_{k,j}^{(t+1)}]^{\lambda_B}) \rightarrow \tilde{\mathbf{w}}_{k,j}^{(t+1)}$. Then, it calculates a b -dimensional vector $\mathbf{v}_m = (v_{m,1}, \dots, v_{m,b})$ with each element calculated as $v_{m,j} = -h_{m,j}(1 - h_{m,j})$, $j \in [1, b]$ and updates the model weights of hidden layer as

$$\begin{aligned}[\tilde{\mathbf{w}}_{j,i}^{(t+1)}] &= [\tilde{\mathbf{w}}_{j,i}^{(t)}] \cdot \prod_{m=1}^S (\Delta_{m,(j,i)})^{\frac{-v_{m,j} x_{m,(2,i)} \eta}{S}} = [\mathbf{w}_{j,i}^{(t)} + \alpha_{j,i} - \\ &\quad \eta \frac{\sum_{m=1}^S -h_{m,j}(1 - h_{m,j}) x_{m,i} \sum_{k=1}^c (y_{m,k} - o_{m,k}) \mathbf{w}_{k,j}^{(t)}}{S}] \\ &= [\mathbf{w}_{j,i}^{(t+1)} + \alpha_{j,i}].\end{aligned}$$

At last, $\mathcal{S}_B$ partially decrypts the encrypted model weights of hidden layer via $\text{PD}(\lambda_B, [\tilde{\mathbf{w}}_{j,i}^{(t+1)}]) \rightarrow [\tilde{\mathbf{w}}_{j,i}^{(t+1)}]^{\lambda_B}$ and sends $\{[\tilde{\mathbf{w}}_{j,i}^{(t+1)}], [\tilde{\mathbf{w}}_{j,i}^{(t+1)}]^{\lambda_B}\}_{j,i=1}^{b,a}$ to $\mathcal{S}_A$.

Step ⑤: After receiving the ciphertexts from $\mathcal{S}_B$, $\mathcal{S}_A$ uses its secret key λ_A to finally recover the model weights of hidden layer by $\text{PD}(\lambda_A, [\tilde{\mathbf{w}}_{j,i}^{(t+1)}]) \rightarrow [\tilde{\mathbf{w}}_{j,i}^{(t+1)}]^{\lambda_A}$, $\text{DD}([\tilde{\mathbf{w}}_{j,i}^{(t+1)}]^{\lambda_A}, [\tilde{\mathbf{w}}_{j,i}^{(t+1)}]^{\lambda_B}) \rightarrow \tilde{\mathbf{w}}_{j,i}^{(t+1)}$.

The above steps are iteratively repeated until the convergence criterion is achieved. When the training procedure is finished, $\mathcal{S}_A$ and $\mathcal{S}_B$ send the perturbed final models of the hidden layer and the output layer to the model requester, respectively. The model requester can easily recover the models by removing the perturbations from the perturbed models. The training process of EPNN is provided in Fig. 2.

4.4 Neural Network Prediction

Our scheme also supports efficient privacy-preserving neural network prediction. Specifically, for a model user MU's encrypted data $[\mathbf{x}_u], [\mathbf{r}_u], \{\prod_{i=1}^a [\alpha_{j,i}]^{-x_{u,i}}\}_{j=1}^b$, $\mathcal{S}_A$ and $\mathcal{S}_B$ cooperatively perform Step ① and ② to obtain the encrypted prediction results $[o_{u,k}]$. $\mathcal{S}_B$ then calculates

$$[o_{u,k}][r_{u,k}] = [o_{u,k} + r_{u,k}],$$

to perturb the prediction results in ciphertexts and subsequently sends it to $\mathcal{S}_A$ after partial decryption $\text{PD}(\lambda_B, [o_{u,k} + r_{u,k}])$. With the secret key λ_A , $\mathcal{S}_A$ can recover the perturbed values via $\text{PD}(\lambda_A, [o_{u,k} + r_{u,k}])$, $\text{DD}([o_{u,k} + r_{u,k}]^{\lambda_A}, [o_{u,k} + r_{u,k}]^{\lambda_B})$. Obviously, MU can easily recover the prediction results $o_{u,k}$ by subtracting $r_{u,k}$ from $o_{u,k} + r_{u,k}$.

Note that, in the above training and prediction phases of EPNN, the model requester, data owner, and model user only submit their data once and are not involved in the iteration process. Thus, very little communication overhead is introduced to the user side. We will confirm this by giving theoretical analysis and experimental results in Section 6. It should be also noted that EPNN is also fit for other activation functions such as Tanh ($f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$), ReLU ($\phi(z) = \max(0, z)$), and

$\text{softmax}(\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}})$ thanks to the homomorphic properties of EPNN. Due to space limitations, we omit the detailed descriptions of these activation functions.

4.5 Discussion of Scope/Limitation

There are some other neural network models, such as convolutional neural networks (CNN) and long and short-term memory neural networks (LSTM). Here, we give a brief discussion of how to apply EPNN to CNN and LSTM.

CNN consists of the input layer, convolutional layer, pooling layer and fully connected layer. To support efficient and privacy-preserving CNN training and prediction on the cloud, EPNN mainly needs to substitute matrix multiplication computations with convolutional operations at the convolutional layer. To achieve this goal, the model requester can perturb the initial weights of the convolutional layer with random values and then send the perturbed weights of the convolutional layer and random values to two CSPs, respectively. After that, the two CSPs can cooperate to complete the convolution operations of the convolution layer based on the encrypted input data. The convolution operation is essentially the dot product of the filter and the input data's local area, and therefore can be converted into a giant matrix multiplication.

LSTM, as a time recurrent neural network, is suitable for processing and predicting events with long intervals and delays in time series due to the introduction of gate mechanisms (i.e., input gate, forget gate, and output gate) to control the flow and loss of features. The LSTM uses hidden states to memorize previous nodes' information. Hidden states combined with the current inputs are used to obtain the gate information. Hidden states and current inputs contain a lot of sensitive information and therefore need to be protected on the cloud. However, it is a challenging issue to use PCTD to support the privacy-preserving calculation of gate information. We leave the design of privacy-preserving LSTM training and prediction scheme as our future work.

5 SECURITY ANALYSIS

In this section, we demonstrate the security of EPNN in terms of data privacy and model privacy in the training and prediction phases, which are usually concerned by the data owner, model requester, and model user. As presented in Section 2, there are four types of adversaries involved in our scheme, as 1) Type-I (i.e., CSP adversary); 2) Type-II (i.e., model requester adversary); 3) Type-III (i.e., data owner adversary); 4) Type-IV (i.e., model user adversary).

5.1 Data Privacy in Neural Network Training

We first prove the privacy of training data, i.e., $\{\mathbf{x}_m, \mathbf{y}_m\}$, in neural network training. Since the entities involved in the training phase include the CSP, the data owner and the model requester, EPNN should protect the data privacy against Type-I, Type-II, and Type-III adversaries, respectively.

Type-I Adversary. Based on the attacks that two CSPs may launch, we will analyze the data privacy in the training phase in the following two cases.

- *Case 1:* For the CSP $\mathcal{S}_A$, it knows the ciphertexts

$$\left([x_{m,i}], \prod_{i=1}^a [\alpha_{j,i}]^{-x_{m,i}}, [\widetilde{\mathbf{wo}}_{k,j}^{(t)}], \zeta_{m,k}, \prod_{k=1}^c (\zeta_{m,k})^{\widetilde{\mathbf{wo}}_{k,j}^{(t)}} \right)$$

and the plaintexts

$$(\beta_{k,j}, \widetilde{\mathbf{wh}}_{j,i}^{(t)}, x_{m,(1,i)})$$

With the secret key λ_A , $\mathcal{S}_A$ may try to recover the sensitive data from the ciphertexts. However, since the PCTD cryptosystem has been proven to be secure against the chosen-plaintexts attack, $\mathcal{S}_A$ cannot obtain any data information, i.e., $\{\mathbf{x}_m, \mathbf{y}_m\}$, from the ciphertexts. Additionally, since $x_{m,i} = x_{m,(1,i)} + x_{m,(2,i)}$, without knowing $x_{m,(2,i)}$, $\mathcal{S}_A$ cannot learn the training data from $x_{m,(1,i)}$ either.

- *Case 2:* Then, we analyze the data privacy against the CSP $\mathcal{S}_B$. $\mathcal{S}_B$ knows the ciphertexts

$$([y_{m,k}], [\beta]_{k,j}, \Delta_{m,(j,i)}, [\widetilde{\mathbf{wh}}_{j,i}^{(t)}])$$

and the plaintexts

$$(x_{m,(2,i)}, \widetilde{\mathbf{wo}}_{k,j}^{(t)}, \alpha_{j,i}, \sum_{i=1}^a x_{m,i} \widetilde{\mathbf{wh}}_{j,i}^{(t)}).$$

Similarly, with the secret key λ_B , it is difficult for $\mathcal{S}_B$ to decrypt ciphertexts. Since $x_{m,(1,i)}$, $\widetilde{\mathbf{wh}}_{j,i}^{(t)}$ are still kept secret from the view of $\mathcal{S}_B$, sensitive data $x_{m,i}$ cannot be inferred from $x_{m,(2,i)}$, and $\sum_{i=1}^a x_{m,i} \widetilde{\mathbf{wh}}_{j,i}^{(t)}$ by $\mathcal{S}_B$.

Therefore, if the two CSPs are non-colluding, the data records $\{\mathbf{x}_m, \mathbf{y}_m\}$ will not be disclosed to Type-I adversary.

Type-II Adversary and Type-III Adversary. The data owner knows the training data, its corresponding labels, i.e., $\{\mathbf{x}_m, \mathbf{y}_m\}$, and the encrypted random values of hidden layer model, i.e., $[\alpha_{j,i}]$. The model requester knows the initial model weights of hidden/output layer, i.e., $\widetilde{\mathbf{wh}}_{j,i}^{(0)}, \widetilde{\mathbf{wo}}_{k,j}^{(0)}$ and the random values of hidden/output layer model, i.e., $\alpha_{j,i}, \beta_{k,j}$. The data owner and model requester only submit their data once in the training process. At the end of training, the model requester is provided with the trained model weights of hidden/output layer, i.e., $\widetilde{\mathbf{wh}}_{j,i}^{(s)}, \widetilde{\mathbf{wo}}_{k,j}^{(s)}$ from two CSPs, where s denotes the number of iterations. Since there is no other interaction between the adversary and the CSP, the data owner and model requester do not have access to data other than they own. Thus, the data privacy holds against Type-II and Type-III adversary.

5.2 Data Privacy in Neural Network Prediction

We then prove the privacy of the prediction data, i.e., $\mathbf{x}_u$, in neural network prediction. Since the entities involved in the prediction phase include the CSP and the model user, where the model user submits the prediction data to the CSP for prediction services. Here, we mainly consider the attack launched by the CSP, which means the data privacy against Type-I adversary.

Type-I Adversary. For CSP $\mathcal{S}_A$, it knows the encrypted data, i.e., $[\mathbf{x}_u], [\mathbf{r}_u], \{\prod_{i=1}^a [\alpha_{j,i}]^{-x_{u,i}}\}_{j=1}^b$, and for CSP $\mathcal{S}_B$, it knows the perturbed values, i.e., $o_{u,k} + r_{u,k}$. Obviously, since

CSP $\mathcal{S}_A$ does not have the secret key λ_B and CSP $\mathcal{S}_B$ does not have the random values $r_{u,k}$, none of the prediction data x_u and the prediction result $o_{u,k}$ can be learned from the ciphertexts and the plaintexts. Thus, the data privacy holds against Type-I adversary.

5.3 Model Privacy in Neural Network Training

The entities involved in the training phase include the CSP, the data owner and the model requester, where the model requester receives the trained models from the CSP. Here, we mainly consider the attack from the CSP and the data owner, which means that model privacy should be protected against Type-I and Type-III adversaries, respectively.

Type-I Adversary. Since the two CSPs are assumed to be non-colluding and the PCTD cryptosystem has been proven to be secure, the ciphertexts cannot be decrypted by any of the CSPs alone. In other words, it is impossible for the two CSPs to retrieve the information of the models, i.e., $\mathbf{wh}_{j,i}^{(t)}$, $\mathbf{wo}_{k,j}^{(t)}$, from the ciphertexts $[\widetilde{\mathbf{wo}}_{k,j}^{(t)}]$, $\prod_{k=1}^c (\zeta_{m,k})^{\widetilde{\mathbf{wo}}_{k,j}^{(t)}}$, $\Delta_{m,(j,i)}$, and $[\widetilde{\mathbf{wh}}_{j,i}^{(t)}]$. Without knowing $\widetilde{\mathbf{wo}}_{k,j}^{(t)}$ and $\alpha_{j,i}$, $\mathcal{S}_A$ cannot obtain any information of the models from $\beta_{k,j}$ and $\widetilde{\mathbf{wh}}_{j,i}^{(t)}$. Similarly, $\mathcal{S}_B$ cannot get any information of the models from $\alpha_{j,i}$ and $\widetilde{\mathbf{wo}}_{k,j}^{(t)}$, either. Thus, the model privacy holds against Type-I adversary.

Type-III Adversary. Similar to the previous analysis, the data owner only knows the training data and its corresponding labels, i.e., $\{x_m, y_m\}$, and encrypted random values of the hidden layer model, i.e., $[\alpha_{j,i}]$. Other information about the models is not available to the data owner as there is no other interaction between the data owner and the CSP once they have uploaded their training data and the corresponding labels to the CSP. And it is impossible to infer anything about the hidden layer model, i.e., $\mathbf{wh}_{j,i}^{(t)}$ simply by knowing the $[\alpha_{j,i}]$. Thus, the model privacy holds against Type-III adversary.

5.4 Model Privacy in Neural Network Prediction

The entities involved in the prediction phase include the CSP and the model user. Here we mainly consider the attack from the CSP and the model user, which means that model privacy should be protected against Type-I and Type-IV adversaries, respectively.

Type-I Adversary. Similar to the analysis in Section 5.3, since the two CSPs are non-colluding, the model, i.e., $\mathbf{wh}_{j,i}^{(t)}$, $\mathbf{wo}_{k,j}^{(t)}$ will not be disclosed to Type-I adversary.

Type-IV Adversary. The model user only receives prediction results, i.e., o_u from the CSPs. Since there is no other interaction between the model user and the CSP, the model user has no chance to obtain the model information, i.e., $\mathbf{wh}_{j,i}^{(t)}$, $\mathbf{wo}_{k,j}^{(t)}$. Thus, the model privacy holds against Type-IV adversary.

In summary, the data privacy and the model privacy of our proposed EPNN scheme can be protected against Type-I, Type-II, Type-III and Type-IV adversaries.

6 PERFORMANCE EVALUATION

In this section, we implement our proposed EPNN scheme and analyze its performance in terms of classification accuracy, computational costs, and communication overhead.

The scheme is implemented in Python, where the encryption primitive is from [21]. Two computer servers equipped with 3.70 GHz Intel Core i9-10900, 128 GB RAM are used as the CSPs, an Android phone with 4GB RAM, Kirin 659 processor acts as the data owner, model requester, and model user, respectively.

Classification Accuracy. We select several real-world datasets^{1 2} (i.e., Iris, Diabetes, Wine) to evaluate the classification accuracy. We select some of the data for each dataset as the training data while the others are used for testing. To process the datasets, we set the multi-layer perception with one hidden layer and a softMax layer (i.e., output layer). Sigmoid is selected as the activation function. For comparison, we implement a conventional MLP scheme without privacy preservation, referred to as n.p.m. We also consider three recent schemes [12], [15], [22], referred to as SecureML, NPMML and PTPNN, respectively. All of these schemes can support privacy-preserving neural network training on the clouds without the assistance of data owners.

Table 1 shows the parameters used in the multi-layer perception and the classification accuracy under different iteration numbers (i.e., from 200 to 400). It is observed that the classification accuracy of our scheme EPNN, NPMML, SecureML, PTPNN and the conventional MLP scheme n.p.m increases with the iteration number. At the same time, there is no significant difference between the five schemes. That is, the classification accuracy of EPNN, NPMML, SecureML and PTPNN are almost identical to that of n.p.m, regardless of the dataset. The accuracy of EPNN, NPMML, SecureML and PTPNN is slightly lower than that of n.p.m, which is caused by truncation errors during the encryptions and decryptions or the approximations to the activation function.

Computational Costs. We then conduct experiments to evaluate the performance of EPNN. In our evaluation, the security parameter κ is set as 512 bits due to security and efficiency. In addition to the datasets of Iris, Diabetes, Wine, we also select MNIST, which contains 60000 images, each of which has 784 features and 10 outputs, to evaluate the computational costs. To process the MNIST dataset, we generate a neural network with one hidden layer of 128 nodes and a SoftMax layer. The learning rate η is set as 0.1, and the batch size, i.e., S , is set as 8, 16, 32, 64, respectively. In addition, since neural network training consumes the most time in privacy-preserving neural networks, we mainly consider the time costs in neural network training.

Fig. 3 summarizes the computational costs of EPNN, NPMML, SecureML and PTPNN under the dataset of Iris, Diabetes, Wine, and MNIST, respectively. It is shown that our scheme can save much more time than NPMML, SecureML and PTPNN in the phase of neural network model training under different datasets. For example, for the dataset of Diabetes, when the batch size is 64 and the iteration number is 100, EPNN only needs 0.63 hours to finish the training process. However, NPMML, SecureML and

1. <http://archive.ics.uci.edu/ml/datasets.php>

2. <https://github.com/susanli2016/Machine-Learning-with-Python/blob/master/diabetes.csv>

TABLE 1
Comparison on Classification Accuracy

	Parameters						Classification accuracy		
	a	b	c	M	S	η	200	300	400
Iris	4	8	3	150	20	0.1	n.p.m	86.84%	97.36%
							EPNN	86.77%	90.00%
							NPMML	86.67%	96.88%
							SecureML	86.33%	96.22%
							PTPNN	86.44%	96.33%
Diabetes	8	10	2	768	20	0.2	n.p.m	67.10%	69.26%
							EPNN	66.85%	69.12%
							NPMML	66.88%	69.17%
							SecureML	66.66%	69.09%
							PTPNN	66.75%	69.12%
Wine	13	16	3	178	20	0.2	n.p.m	90.74%	92.59%
							EPNN	88.89%	91.67%
							NPMML	88.89%	91.48%
							SecureML	88.33%	91.11%
							PTPNN	88.52%	91.30%

PTPNN need about 2.23 hours, 5.78 hours and 4.25 hours, respectively. For the dataset of MNIST, when the number of batch sizes is 64, EPNN, NPMML, SecureML and PTPNN need about 1.27 hours, 21.93 hours, 32.07 hours and 29.54 hours, respectively. Therefore, we can figure out that our scheme is more efficient than NPMML, SecureML and PTPNN.

Communication Overhead. We last evaluate the communication overhead of our scheme. Since NPMML does not consider the privacy of trained models, we omit the communication overhead on MR for a fair comparison. The size of plaintexts is not taken into account, as it is negligible compared with the ciphertexts.

Table 2 summarizes the communication complexity of our scheme EPNN, NPMML, SecureML and PTPNN in the

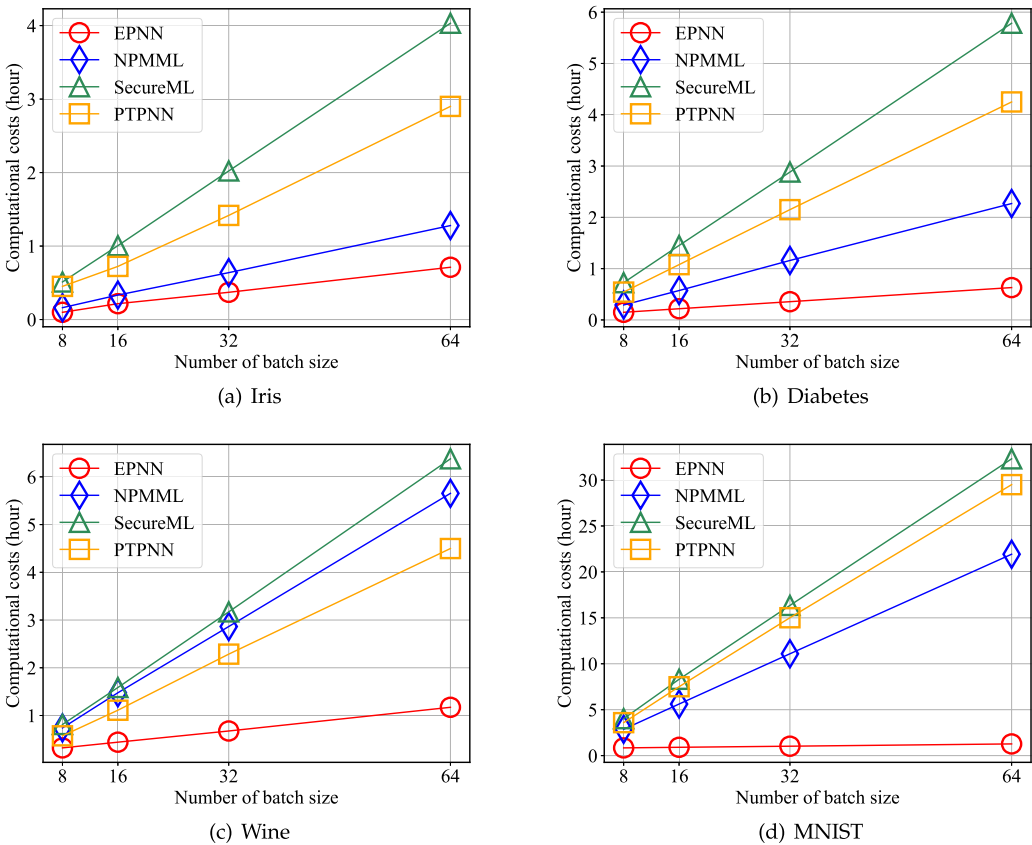


TABLE 2

Comparison of Communication Complexity, Where D.S Denotes Data Submission, N.T Denotes Neural Network Training, and N.P Denotes Neural Network Prediction, a Denotes the Dimension of the Data, b Denotes the Number of Nodes in the Hidden Layer, c Denotes the Dimension of the Output Layer, $|C|$ Denotes the Size of Ciphertexts, S Denotes the Batch Size, and T Denotes the Iteration Number

		EPNN	NPMML
D.S	DO	$(a + b + c) C $	$(a + c) C $
	MU	$(a + b + c) C $	$(a + c) C $
N.T	CSP	$(S(ab + 3b + c) + 2b(a + c))T C $	$(S(2ab + bc + a + b + c) + bc)T C $
N.P	CSP	$(2b + 2c) C $	$(ab + a + 2c) C $
		SecureML	PTPNN
D.S	DO	$(a + c)$	$(a + c)$
	MU	$(a + c)$	$(a + c)$
N.T	CSP	$(2ab + 2bc + 2a + 14b + 8c)ST C $	$(2ab + 2bc + 2a + 8b + 4c)ST C $
N.P	CSP	$(2a + 10b + 6c) C $	$(2a + 4b + 2c) C $

phases of data submission, neural network training, and neural network prediction. It is observed that our scheme performs a little worse on the user side but is far more communication-efficient on the CSPs. In Table 3, we give the communication costs in different phases under different datasets. It can be seen that our scheme can significantly reduce the communication overhead on the CSPs. Specifically, compared with NPMML, EPNN can save about half of communication overhead for the datasets of MNIST, Wine, and save about 42% of communication overhead for the datasets of Iris, Diabetes in the phase of neural network training. Compared with SecureML, EPNN can save about 50%, 66%, 69%, and 76% communication overhead for the datasets of MNIST, Wine, Diabetes, and Iris, respectively, while compared to PTPNN, it can save about 50%, 61%, 62%, and 69% communication overhead respectively. In the phase of neural network prediction, the communication overhead between the two clouds of EPNN is also much less than that of NPMML, SecureML and PTPNN. Although users' communication overhead in EPNN is a little more than others, it is acceptable because the users only submit their data once and will not participate in the subsequent training and prediction phases.

7 RELATED WORK

In the past decades years, the privacy of machine learning, such as privacy-preserving k-means clustering [23] [24], privacy-preserving decision trees [25] [26], privacy-preserving linear regression [27] [28] and privacy-preserving logistic regression [29], have received considerable attention. In this paper, we mainly focus on privacy-preserving neural network training and prediction.

Privacy-Preserving Neural Network Training. Shokri and Shmatikov [30] proposed a privacy-preserving deep learning system, allowing multiple parties to train a neural network model jointly. Instead of sharing their private data, all participants share the variation of partial coefficients. But their method is only suitable for horizontally partitioned data and lacks formal data security guarantees. After that, Zhao et al. [31] designed SecProbe, a privacy-preserving collaborative deep learning system, which first considers the existence of unreliable participants. The authors suggested an exponential mechanism to reduce the influence of unreliable participants. SecProbe can effectively obtain an accurate model while protecting the privacy of all participants. Nandakumar et al. [32] presented a feasible way to non-interactively train a deep neural network on encrypted

TABLE 3

Comparison of Communication Overhead, Where D.S Denotes Data Submission, N.T Denotes Neural Network Training, N.P Denotes Neural Network Prediction, and T Denotes the Iteration Number

		MNIST				Iris			
		EPNN	NPMML	SecureML	PTPNN	EPNN	NPMML	SecureML	PTPNN
D.S	DO	0.23 MB	0.19 MB	0.10 KB	0.10 KB	3.75 KB	1.75 KB	0.001KB	0.001KB
	MU	0.23 MB	0.19 MB	0.10 KB	0.10 KB	3.75 KB	1.75 KB	0.001KB	0.001KB
N.T	CSP	$1624T$ MB	$3171T$ MB	$3230T$ MB	$3217T$ MB	$0.95T$ MB	$1.62T$ MB	$4.00T$ MB	$3.06T$ MB
N.P	CSP	0.07 MB	24.70 MB	0.71 MB	0.51 MB	5.50 KB	10.50 KB	26.50 KB	11.50 KB
		Diabetes				Wine			
		EPNN	NPMML	SecureML	PTPNN	EPNN	NPMML	SecureML	PTPNN
D.S	DO	5.00 KB	2.50 KB	0.001 KB	0.001 KB	8.00 KB	4.00 KB	0.002 KB	0.002 KB
	MU	5.00 KB	2.50 KB	0.001 KB	0.001 KB	8.00 KB	4.00 KB	0.002 KB	0.002 KB
N.T	CSP	$1.80T$ MB	$3.13T$ MB	$5.81T$ MB	$4.75T$ MB	$4.17T$ MB	$7.76T$ MB	$12.28T$ MB	$10.59T$ MB
N.P	CSP	6.00 KB	23.00 KB	32.00 KB	15.00 KB	9.50 KB	56.75 KB	51.00 KB	24.00 KB

data. Using minimal precision drop, proper data representation and resolution, and smart data packing improves the computational efficiency of cryptographic training. However, their scheme is essentially designed based on fully homomorphic encryption (FHE), which will impose huge computational costs on the service provider. In addition, He et al. [33] proposed a cloud-based privacy-preserving collaborative neural network training scheme, TransNet, that can support arbitrarily partitioned datasets by adding a transformed layer. However, TransNet focuses on how to protect the privacy of arbitrary partitioned datasets in the training process, ignoring the model privacy on the cloud. Zheng et al. [34] extended the privacy-preserving neural network learning paradigm to an industrial scale, where parties conduct calculations on private data locally, and other expensive computations are transferred to servers with high computational capability.

Privacy-Preserving Neural Network Prediction. Barni et al. [35] first proposed a privacy-preserving neural network prediction scheme in which the neural network can run on the encrypted data to get prediction results. Additionally, the model is also protected, for it is the private asset of a specific organization or individual. However, their method has high computation and communication overhead since model users need to stay online to decrypt intermediate results. Using leveled homomorphic encryption (LHE) [36] [37] [38], Gilad-Bachrach et al. [39] designed a scheme to convert neural networks into CryptoNets which can perform on encrypted data and return the encrypted result. LHE is only suitable for arithmetic operations (e.g., addition and multiplication) and is not easy to apply directly to the activation function. After that, Liu et al. [40] converted the existing neural network into MiniONN to support privacy-preserving prediction with reasonable efficiency. Combining single-instruction-multiple-data (SIMD) [41], asynchronous computation and garbled circuits, Li et al. [42] proposed an outsourcing scheme for privacy-preserving convolutional neural network prediction, in which the server is unable to learn intermediate results and models. Compared with MiniONN [40], it is more efficient. Ma et al. [17] designed a two-server-based scheme in which the model owner can safely outsource the existing model to the servers. Subsequently, the two servers will provide prediction services on encrypted data for public users. Xie et al. [16] put forward a practical solution for secure DNN inference, BAYHENN, which has greatly improved the end-to-end latency. Specifically, the core strategy of BAYHENN is to combine homomorphic encryption and Bayesian neural networks to protect the original data and model weights, respectively. Additionally, Liu et al. [18] presented a lightweight, secure neural network inference system, Leia, for edge computing. Two non-colluding edge servers perform Binary Neural Network (BNN) inference on encoded user data and models, with low inference overhead.

Recently, a large number of privacy-preserving federated learning schemes have also been proposed. Based on a chained secure multiparty computing technique, Li et al. [43] introduced a novel privacy-preserving federated learning framework called chain-PPFL. The communication and computational complexity of chain-PPFL is much lower than that of existing secure multiparty computing-based

schemes, making them suitable for deployment in practical scenarios. Yin et al. [44] designed a new hybrid privacy-preserving approach to address the risk of data and content leakage during federated learning. Specifically, they use an advanced functional encryption algorithm to protect the privacy of data and model weights in the weighted summation process. In addition, Zhao et al. [45] presented an anonymous and privacy-preserving federated learning scheme for industrial big data mining. Their scheme minimizes privacy leakage by sharing a smaller number of parameters between the server and each participant. However, the above schemes are all concerned with privacy-preserving neural network training in federated learning scenarios, where the data owner needs to stay online and cooperate with the cloud to train models. In our scheme, all users are offline after submitting their data, and two CSPs undertake the task of neural network training. Therefore, our scheme is computationally and communication efficient on the user side.

8 CONCLUSION

In this paper, we consider the privacy and efficiency issues in cloud-based neural network applications and present an efficient and privacy-preserving neural network training and prediction scheme. Our scheme relieves users from time-consuming neural network training and prediction processes and enables neural networks to be performed in cloud environments in an efficient and privacy-preserving manner. Detailed security analysis demonstrates the privacy of data and trained models. Extensive experiments based on real-world datasets show our scheme has high computational efficiency and low communication overhead. We believe this work can deal with users' privacy concerns and make cloud-based privacy-preserving neural networks practical in real-world applications. In the future work, we will focus on the verifiability issue in neural network and try to design a verifiable privacy-preserving neural network scheme.

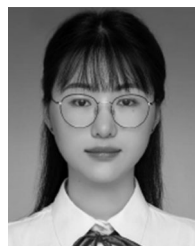
REFERENCES

- [1] O. Atan, J. Jordon, and M. van der Schaar, "Deep-treat: Learning optimal personalized treatments from observational data using neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 2071–2078.
- [2] J. Yang, D. She, and M. Sun, "Joint image emotion classification and distribution learning via deep convolutional neural network," in *Proc. Int. Joint Conf. Artif. Intell.*, 2017, pp. 3266–3272.
- [3] W.-D. Xi, L. Huang, C.-D. Wang, Y.-Y. Zheng, and J. Lai, "BPAM: Recommendation based on BP neural network with attention mechanism," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 3905–3911.
- [4] Q. Zhang, C. Wang, H. Wu, C. Xin, and T. V. Phuong, "Gelu-Net: A globally encrypted, locally unencrypted deep neural network for privacy-preserved learning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2018, pp. 3933–3939.
- [5] M. Nasr, R. Shokri, and A. Houmansadr, "Machine learning with membership privacy using adversarial regularization," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 634–646.
- [6] N. Phan et al., "Heterogeneous gaussian mechanism: Preserving differential privacy in deep learning with provable robustness," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 4753–4759.
- [7] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 1322–1333.

- [8] P. Xie, M. Bilenko, T. Finley, R. Gilad-Bachrach, K. Lauter, and M. Naehrig, "Crypto-nets: Neural networks over encrypted data," 2014, *arXiv:1412.6181*.
- [9] P. Li, J. Li, Z. Huang, T. Li, C.-Z. Gao, S.-M. Yiu, and K. Chen, "Multi-key privacy-preserving deep learning in cloud computing," *Future Gener. Comput. Syst.*, vol. 74, pp. 76–85, 2017.
- [10] J. Yuan and S. Yu, "Privacy preserving back-propagation neural network learning made practical with cloud computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, pp. 212–221, Jan. 2014.
- [11] D. Boneh, E.-J. Goh, and K. Nissim, "Evaluating 2-DNF formulas on ciphertexts," in *Proc. Theory Cryptogr. Conf.*, 2005, pp. 325–341.
- [12] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. on Secur. Privacy*, 2017, pp. 19–38.
- [13] P. Mohassel and P. Rindal, "ABY3: A mixed protocol framework for machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 35–52.
- [14] N. Agrawal, A. Shahin Shamsabadi, M. J. Kusner, and A. Gascón, "Quotient: Two-party secure neural network training and prediction," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1231–1247.
- [15] T. Li, J. Li, X. Chen, Z. Liu, W. Lou, and Y. T. Hou, "NPMML: A framework for non-interactive privacy-preserving multi-party machine learning," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 6, pp. 2969–2982, Nov./Dec. 2020.
- [16] P. Xie, B. Wu, and G. Sun, "BAYHENN: Combining Bayesian deep learning and homomorphic encryption for secure DNN inference," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 4831–4837.
- [17] X. Ma, X. Chen, and X. Zhang, "Non-interactive privacy-preserving neural network prediction," *Inf. Sci.*, vol. 481, pp. 507–519, 2019.
- [18] X. Liu, B. Wu, X. Yuan, and X. Yi, "Leia: A lightweight cryptographic neural network inference system at the edge," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 237–252, 2022.
- [19] C. Miao, L. Su, W. Jiang, Y. Li, and M. Tian, "A lightweight privacy-preserving truth discovery framework for mobile crowd sensing systems," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2017, pp. 1–9.
- [20] C. Zhang, L. Zhu, C. Xu, X. Liu, and K. Sharif, "Reliable and privacy-preserving truth discovery for mobile crowdsensing systems," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1245–1260, May/Jun. 2021.
- [21] X. Liu, B. Qin, R. H. Deng, R. Lu, and J. Ma, "A privacy-preserving outsourced functional computation framework across large-scale multiple encrypted domains," *IEEE Trans. Comput.*, vol. 65, no. 12, pp. 3567–3579, Dec. 2016.
- [22] Z. Ge, Z. Zhou, D. Guo, and Q. Li, "Practical two-party privacy-preserving neural network based on secret sharing," 2021, *arXiv:2104.04709*.
- [23] P. Bunn and R. Ostrovsky, "Secure two-party k-means clustering," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2007, pp. 486–497.
- [24] M. Upmanyu, A. M. Namboodiri, K. Srinathan, and C. Jawahar, "Efficient privacy preserving k-means clustering," in *Proc. Pacific-Asia Workshop Intell. Secur. Inform.*, 2010, pp. 154–166.
- [25] J. Vaidya, C. Clifton, M. Kantarcioglu, and A. S. Patterson, "Privacy-preserving decision trees over vertically partitioned data," *ACM Trans. Knowl. Discov. Data*, vol. 2, pp. 1–27, 2008.
- [26] P. K. Fong and J. H. Weber-Jahnke, "Privacy preserving decision tree learning using unrealized data sets," *IEEE Trans. Knowl. Data Eng.*, vol. 24, no. 2, pp. 353–364, Feb. 2012.
- [27] W. Du, Y. S. Han, and S. Chen, "Privacy-preserving multivariate statistical analysis: Linear regression and classification," in *Proc. SIAM Int. Conf. Data Mining*, 2004, pp. 222–233.
- [28] A. P. Sanil, A. F. Karr, X. Lin, and J. P. Reiter, "Privacy preserving regression modelling via distributed computation," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2004, pp. 677–682.
- [29] K. Chaudhuri and C. Monteleoni, "Privacy-preserving logistic regression," in *Proc. Adv. Neural Inf. Process. Syst.*, 2008, pp. 289–296.
- [30] R. Shokri and V. Shmatikov, "Privacy-preserving deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 1310–1321.
- [31] L. Zhao, Q. Wang, Q. Zou, Y. Zhang, and Y. Chen, "Privacy-preserving collaborative deep learning with unreliable participants," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 1486–1500, 2020.
- [32] K. Nandakumar, N. Ratha, S. Pankanti, and S. Halevi, "Towards deep neural network training on encrypted data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2019, pp. 40–48.
- [33] Q. He, W. Yang, B. Chen, Y. Geng, and L. Huang, "TransNet: Training privacy-preserving neural network over transformed layer," *Proc. VLDB Endowment*, vol. 13, pp. 1849–1862, 2020.
- [34] L. Zheng, C. Chen, Y. Liu, B. Wu, X. Wu, L. Wang, L. Wang, J. Zhou, and S. Yang, "Industrial scale privacy preserving deep neural network," 2020, *arXiv:2003.05198*.
- [35] M. Barni, C. Orlandi, and A. Piva, "A privacy-preserving protocol for neural-network-based computation," in *Proc. Workshop Multimedia Secur.*, 2006, pp. 146–151.
- [36] N. Dowlin, R. Gilad-Bachrach, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Manual for using homomorphic encryption for bioinformatics," *Proc. IEEE*, vol. 105, pp. 552–567, Mar. 2017.
- [37] J. W. Bos, K. Lauter, J. Loftus, and M. Naehrig, "Improved security for a ring-based fully homomorphic encryption scheme," in *Proc. IMA Int. Conf. Cryptogr. Coding*, 2013, pp. 45–64.
- [38] A. López-Alt, E. Tromer, and V. Vaikuntanathan, "On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption," in *Proc. Annu. ACM Symp. Theory Comput.*, 2012, pp. 1219–1234.
- [39] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 201–210.
- [40] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via miniONN transformations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.
- [41] N. P. Smart and F. Vercauteren, "Fully homomorphic simd operations," *Des., Codes Cryptogr.*, vol. 71, pp. 57–81, 2014.
- [42] M. Li, S. S. Chow, S. Hu, Y. Yan, S. Chao, and Q. Wang, "Optimizing privacy-preserving outsourced convolutional neural network predictions," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1592–1604, May/Jun. 2020.
- [43] Y. Li, Y. Zhou, A. Jolfaei, D. Yu, G. Xu, and X. Zheng, "Privacy-preserving federated learning framework based on chained secure multiparty computing," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6178–6186, Apr. 2021.
- [44] L. Yin, J. Feng, H. Xun, Z. Sun, and X. Cheng, "A privacy-preserving federated learning for multiparty data sharing in social IoTs," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 3, pp. 2706–2718, Jul.–Sep. 2021.
- [45] B. Zhao, K. Fan, K. Yang, Z. Wang, H. Li, and Y. Yang, "Anonymous and privacy-preserving federated learning with industrial big data," *IEEE Trans. Ind. Informat.*, vol. 17, no. 9, pp. 6314–6323, Sep. 2021.



Chuan Zhang received the PhD degree in computer science and technology from the Beijing Institute of Technology, Beijing, China, in 2021. He is currently an assistant professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include secure data services in applied cryptography, cloud security, blockchain, and machine learning.



Chenfei Hu received the BS degree from Nanchang University, in 2021. She is currently working towards the master's degree with the School of Computer Science and Technology, Beijing Institute of Technology. Her research interests include applied cryptography, cloud security, and machine learning.



Tong Wu received the PhD degree in computer science from the University of Wollongong, Australia, in 2020. She is currently a postdoctoral research fellow with the School of Cyberspace Science and Technology, Beijing Institute of Technology. Her research interests include applied cryptography, cloud security, security and privacy in blockchain.



Liehuang Zhu (Senior Member, IEEE) received the PhD degree in computer science from the Beijing Institute of Technology, Beijing, China, in 2004. He is currently a professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include security protocol analysis and design, group key exchange protocols, wireless sensor networks, and cloud computing.



Ximeng Liu (Senior Member, IEEE) received the BSc degree in electronic engineering from Xidian University, Xi'an, China, in 2010, and the PhD degree in cryptography from Xidian University, China, in 2015. He was the research assistant with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore from 2013 to 2014, and research fellow with the School of Information System, Singapore Management University, Singapore. He is currently a professor with Fuzhou University. His

research interests include cloud security, applied cryptography and Big Data security.

► **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**